PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
46504-04 · Construção de Software · Turma 330 · 2026/2
Prof. Michael da Costa Mora

PROJETO INTEGRADOR · PACOTE 3

PulsePass

Checkpoint do Desenvolvimento

Estado do código, documentação atualizada dos Pacotes 1 e 2 e evidências de teste

Integrantes: Gabriel Kowaleski e Mayra Bordin
Repositório: **REPO_URL**
Entrega: 23 de novembro de 2026

1. Resumo do checkpoint

O repositório contém a fatia vertical **R1** do backend do PulsePass: do catálogo de eventos até o ingresso validado na portaria, passando por reserva atômica, pagamento idempotente, Outbox e Kafka. Tudo sobe com `docker compose up -d --wait` (PostgreSQL, Redis, Kafka e Kafka UI, Keycloak, API Gateway, três serviços de negócio e o simulador de pagamento) e o script `scripts/smoke.sh` percorre o fluxo completo pelo gateway, autenticando no Keycloak.

Indicador	Resultado
Testes automatizados	**52**, todos passando: 20 unitários, 25 de integração com PostgreSQL, Redis e Kafka reais (Testcontainers), 3 do simulador e 4 do gateway.
Cobertura de linhas do domínio	Inventário 100%, Checkout 100%, Emissão 93% (meta: 70%, verificada no build).
Teste de mutação (PIT)	100% dos mutantes mortos nos três domínios (meta: 60%).
Smoke ponta a ponta	Passa contra os 10 contêineres reais, incluindo RBAC, idempotência, ingresso em PDF e validação do QR.
Histórias de R1 e R2 entregues	19 de 21 atendidas; 2 parciais (US01, sem auto-cadastro; US14, sem WebSocket).
Requisitos não funcionais	11 atendidos, 7 parciais, 5 pendentes (seção 4).

> **Limites honestos.** A CI (GitHub Actions) foi escrita, mas só roda depois da publicação do repositório. Testes de carga (k6), de réplicas e de falha do emissao-service não foram executados. A cobertura de 70% mede o pacote `domain` (máquinas de estado e regras puras); a orquestração dos serviços é exercida pelos testes de integração, mas não entra nesse percentual.

1.1 Como reproduzir

```
cp .env.example .env
docker compose up -d --build --wait
./scripts/smoke.sh
./mvnw verify                       # unitários + integração + formatação + cobertura do domínio
./mvnw -Pmutacao -pl services/inventario-service test-compile org.pitest:pitest-
maven:mutationCoverage -DskipITs
docker compose down -v
```

Pré-requisitos: Docker com Compose e Java 21 (o Maven vem pelo wrapper). O `README.md` do repositório traz os detalhes, as portas e os testes que sustentam cada requisito.

2. Estado do backlog (Pacote 1)

Status das histórias da release R1 e das de R2 que foram entregues junto, com a evidência de cada uma. Histórias não citadas individualmente estão no último grupo.

História	Descrição	Status	Evidência
US01	Criar conta com e-mail e senha	Parcial	Contas de teste no realm do Keycloak; o auto-cadastro está desligado (`registrationAllowed=false`).
US02	Entrar e receber token de acesso	Atendido	`smoke.sh` obtém tokens no Keycloak e os usa pelo gateway.
US04	Papel adequado em cada rota (RBAC)	Atendido	401 e 403 testados em todos os serviços (`ConsumidorEApiIT` , `CheckoutIT` , `EmissaoIT` , `RotasTest`) e no smoke.
US07	Cadastrar evento	Atendido	`ConsumidorEApiIT` , `smoke.sh` .
US08	Criar lotes com preço, quantidade e período	Atendido	`Fixtures` nos testes e `smoke.sh` .
US09	Setores e assentos numerados (R2)	Atendido	Setor marcado cria os assentos; usado em todos os testes de reserva.
US11	Listar eventos	Atendido	Catálogo público com cache Redis; `smoke.sh` .
US12	Detalhes, lotes e disponibilidade	Atendido	`GET /eventos/{id}` ; disponibilidade por assento livre ou por contadores do lote.
US14	Mapa de assentos livre, reservado e vendido	Parcial	API pronta (`GET /eventos/{id}/assentos`); a atualização por WebSocket e a SPA ficam para o Pacote 4.
US15	Reserva atômica por 10 minutos	Atendido	`ReservaConcorrenteIT` : 500 reservas simultâneas, 1 sucesso e 499 conflitos.
US16	Liberar reservas não pagas	Atendido	`ExpiracaoEOutboxIT` .
US17	Cancelar reserva (R2)	Atendido	`ConsumidorEApiIT.clienteCancelaAReserva...` .
US19	Revisar pedido com itens e total	Atendido	`GET /pedidos/{id}` ; `CheckoutIT` .
US20	Pagar o pedido	Atendido	`CheckoutIT` , `smoke.sh` .
US21	Clique repetido não cobra duas vezes	Atendido	`CheckoutIT` : 1 chamada ao gateway; 20 chamadas concorrentes, 1 pagamento.
US23	Pagamento recusado e nova tentativa (R2)	Atendido	`CheckoutIT` : recusa volta a RESERVADO e a segunda tentativa paga.
US24	Ingresso em PDF com QR Code	Atendido	`EmissaoIT` (PDF lido pelo PDFBox) e `smoke.sh` .
US26	Ver todos os meus ingressos	Atendido	`GET /ingressos` (API).
US28	Operador lê o QR e vê se é válido	Atendido	`EmissaoIT` , `smoke.sh` .
US29	Ingresso usado é recusado (R2)	Atendido	Resposta 409 com o horário da primeira leitura; 50 leituras simultâneas liberam 1.
US31	Total vendido e ocupação por lote	Atendido	`GET /eventos/{id}/vendas` , só para o dono do evento (`ConsumidorEApiIT`).

História	Descrição	Status	Evidência
US03, US05, US06, US10, US13, US18, US22, US25, US27, US30, US32 a US39	Renovação automática da sessão, cadastro de organizador, auditoria, publicar evento, busca, fila virtual, publicação por Outbox como história própria, e-mail, tempo real, painéis	Pendente	Fora da fatia R1. US22 está atendida na prática (Outbox em todos os serviços), mas sem história própria no código.

3. Divergências em relação ao Pacote 2

O desenho do Pacote 2 foi seguido, com as diferenças abaixo. Todas estão registradas aqui e, quando afetam contratos, no documento atualizado do Pacote 2 (seção 8).

Seção do Pacote 2	Desenhado	Como ficou
2.3.2 Checkout	`IdempotencyFilter` como filtro servlet	A lógica está em `PagamentoService` (TX1, gateway fora de transação, TX2). Mesmas regras; chamada concorrente com a mesma chave enquanto a primeira roda recebe 409.
2.3.2 Checkout	Falha do gateway	Responde 503; o pagamento falho passa a ter chave própria (`#falha:id`) para a chave do cliente poder ser reutilizada. Recusa responde 402.
2.3.3 Emissão	`PdfStorage` e `NotificacaoWorker` / `EmailSender`	O PDF é gerado sob demanda, sem armazenamento (serviço continua sem estado). E-mail e a tabela `NOTIFICACAO` ficam para o Pacote 4 (US25 é R2).
2.2 Contêineres	iam-service, telemetria-service, realtime-service e SPA	Não implementados no Pacote 3. Os papéis vêm direto do Keycloak (`realm_access.roles`), e cada serviço valida o JWT pelo JWKS.
2.2 Contêineres	API Gateway com rate limit	O gateway valida JWT e roteia sob o prefixo `/api` ; o rate limit não foi implementado.
2.3.1 Inventário	`FilaController` e fila virtual	Não implementados. O Redis é usado só para o cache do catálogo e para o `SeatLock` .
3 Dados	Tabela de Outbox	`outbox_event` guarda `topic` , `msg_key` e o envelope completo em `body` . A reserva ganhou `suspensa_ate` (ADR-008) e o lote tem `check` contra sobre-venda. O ingresso guarda `evento_nome` e `descricao` copiados do pedido.
4 Kafka	Criação de tópicos	Um contêiner `kafka-init` cria os tópicos com as partições da seção 4.2 e a criação automática está desligada. Sem isso, o consumidor que subia primeiro fazia o Kafka criar o tópico com 1 partição (defeito encontrado no smoke). Existem os tópicos de reserva, assento, pedido, ingresso e três DLQs; os de IAM e telemetria não foram criados.
4 Kafka	DLQ	Três tentativas com intervalo de 500 ms e depois o tópico `<tópico>.<grupo>.dlq` , na biblioteca `commons` .
5 Pipeline	Etapas	Workflow `.github/workflows/ci.yml` com lint, testes, smoke, varredura de segredos, PIT (perfil Maven `mutacao` , no push em develop e à noite) e publicação no GHCR. Compose sem OpenTelemetry, Prometheus, Grafana e Mailpit.
1.1 Stack	Versões	Spring Boot 3.5.16, Spring Cloud 2025.0.3, Resilience4j 2.4.0 (uso programático), PDFBox 3.0.8, ZXing 3.5.4, Testcontainers 2.x, Keycloak 26.0.

4. Requisitos não funcionais: estado e evidência

RNF	Requisito	Estado	Evidência
RNF01	Zero sobre-venda	Atendido	`ReservaConcorrenteIT` (assento e lote); restrição `check` no banco como última barreira.
RNF02	Expiração em 10 min + 30 s	Atendido	`ExpiracaoEOutboxIT` (TTL reduzido a 2 s no teste); suspensão limitada a 30 s.
RNF03	Pagamento idempotente	Atendido	`CheckoutIT`.
RNF04	Transactional Outbox	Atendido	`ExpiracaoEOutboxIT`: com o Kafka pausado o evento fica pendente e sai quando ele volta.
RNF05	Consumo idempotente	Atendido	Duplicata de `PedidoPago`, `ReservaCriada` e `ReservaExpirada` testada nos três serviços.
RNF06	Catálogo p95 ≤ 200 ms a 1.000 req/s	Parcial	Cache Redis implementado; teste de carga (k6) não executado.
RNF07	Reserva p95 ≤ 500 ms a 200/s	Parcial	Filtro Redis implementado; teste de carga não executado.
RNF08	Ingresso em até 30 s (p95)	Pendente	Depende do telemetria-service. Na prática o ingresso aparece em poucos segundos no smoke.
RNF09	Réplicas sem erro funcional	Parcial	Sem estado em memória; Outbox e expiração usam `SKIP LOCKED`; teste com 2 réplicas não executado.
RNF10	Timeout, retry e circuit breaker	Atendido	`CheckoutIT`: 3 tentativas com backoff, 503, pedido volta a RESERVADO.
RNF11	Emissão fora não impede a venda	Parcial	O desenho vale (Kafka retém); não há teste derrubando o emissao-service.
RNF12	JWT stateless (15 min, refresh 7 dias)	Parcial	401 sem token testado; tempos configurados no realm, mas a renovação não foi testada.
RNF13	RBAC em todas as rotas	Atendido	Testes por papel nos serviços e no gateway.
RNF14	QR assinado	Atendido	`DominioTest e EmissaoIT`: adulterado ou de outro segredo é rejeitado.
RNF15	LGPD	Parcial	Senhas só no Keycloak; os serviços guardam apenas o identificador do usuário; sem revisão formal de logs.
RNF16	Observabilidade	Pendente	OpenTelemetry, Prometheus, Grafana e correlation-id ficam para o Pacote 4.
RNF17	Cobertura > 70% e Testcontainers	Atendido	Domínio com 100%, 100% e 93% de linhas; integração com PostgreSQL, Redis e Kafka reais.
RNF18	Lint e arquitetura documentada	Atendido	Spotless (google-java-format) falha o build; C4 e ADRs no Pacote 2.
RNF19	Ambiente com um comando	Atendido	`docker compose up -d --wait` sobe 10 contêineres saudáveis; verificado nesta máquina.
RNF20	CI bloqueia PR vermelho	Parcial	Workflow escrito e validado como YAML; ainda não executou no GitHub porque o repositório não foi publicado.
RNF21	Compra em até 5 telas	Pendente	Sem SPA no Pacote 3.
RNF22	Acessibilidade WCAG AA	Pendente	Sem SPA no Pacote 3.
RNF23	Fila virtual por ordem de chegada	Pendente	Não implementada.

5. Atualizações do Pacote 1

O Pacote 1 entregue continua válido. As atualizações são estas:

- **Backlog:** o status por história está na seção 2. Nenhuma história foi removida; as de R2 e R3 não citadas seguem como candidatas.
- **Regras esclarecidas durante a construção:** (1) o pedido nasce CRIADO e vira RESERVADO quando a reserva é confirmada pelo evento do Inventário; (2) pagamento recusado ou falha do gateway devolvem o pedido a RESERVADO, para nova tentativa enquanto a reserva estiver ativa; (3) a capacidade de um lote é controlada por contadores (`qtd_reservada` e `qtd_vendida`); (4) a expiração da reserva fica suspensa por até 30 s durante o pagamento.
- **Processo com IA (SDD):** as três funcionalidades críticas têm spec em `/specs` (reserva atômica, pagamento idempotente, ingresso com QR), escritas antes do código. As regras dos agentes estão versionadas em `.claude/CLAUDE.md` . O código foi gerado com apoio do Claude Code e revisado pelo grupo; os testes de integração e o smoke serviram de contrapeso e acharam defeitos reais, descritos abaixo.
- **Ferramentas efetivamente usadas:** Java 21, Spring Boot 3.5, Maven, Testcontainers, PIT, GitHub Actions, Docker Compose, Keycloak. Figma, GitHub Projects e Structurizr continuam previstos para os próximos pacotes.
- **Escopo:** a SPA e os mockups do Pacote 1 não foram implementados neste checkpoint (decisão do grupo: só APIs).

5.1 Defeitos achados pelos testes e corrigidos

- **Chave de idempotência reutilizada após falha do gateway:** o pagamento falho mantinha a chave e a nova tentativa violava a unicidade (achado em `CheckoutIT`).
- **Erros 409, 422 e 404 virando 403:** o redirecionamento para `/error` caía na regra de negação do Spring Security. Os testes com MockMvc não veem isso; o smoke contra o Compose real encontrou.
- **Tópico criado com 1 partição:** corrida de criação de tópicos entre serviços; corrigida com o contêiner `kafka-init` e criação automática desligada.
- **Limite do segredo do QR:** mutante sobrevivente (32 contra 31 caracteres) fechado com teste de fronteira.

6. Atualizações do Pacote 2

O documento do Pacote 2 foi regenerado como `PulsePass_Pacote2_atualizado.pdf`, com a nova seção 8 (atualizações do Pacote 3) e a tabela de divergências da seção 3 deste documento. O arquivo originalmente entregue foi mantido como `PulsePass_Pacote2_entregue.pdf`.

O Pacote 1 original não tem fonte editável neste repositório, por isso ele é mantido como entregue (`PulsePass_Pacote1_entregue.pdf`) e suas atualizações estão na seção 5 deste documento.

7. Próximos passos (Pacote 4)

- SPA responsiva (catálogo, mapa de assentos, checkout, meus ingressos, leitor de QR) e acessibilidade (RNF21, RNF22).
- `iam-service`, `telemetria-service` e `realtime-service`; e-mail com Mailpit; fila virtual (RNF23).
- Observabilidade com OpenTelemetry, Prometheus e Grafana (RNF16); correlation-id.
- Testes de carga (k6) e de réplicas; teste derrubando o emissao-service; verificação da CI no GitHub.
- Auto-cadastro de clientes no Keycloak (US01) e publicação de eventos (US10).